

Pricing Barrier Options under Heston Stochastic Volatility:
A Python Implementation of the Method of Lines and 2D COS
Method,
Cross-Validated against Monte Carlo and Chiarella, Kang & Meyer
(2012)

Now Extended to American Exercise via Bermudan-COS

Replication report

September 11, 2026

Abstract

This note reports on a Python implementation of independent pricing engines for up-and-out barrier call options under Heston stochastic volatility (optionally with Merton/Bates log-normal jumps): a *method-of-lines* (MOL) finite-difference PDE solver ported from the Fortran reference code, a *2D COS* (Fourier-cosine) expansion pricer ported from the MATLAB reference code, an independent *Monte Carlo* (MC) simulator, and a new *Bermudan-COS American* extension of the 2D COS pricer built for this report. All are cross-validated against each other and against the published numerical examples of Chiarella, Kang & Meyer (2012, *Comput. Math. Appl.* 64, 2034–2048) – the paper this codebase’s method of lines implements. We report replication accuracy, discuss a grid-sensitivity issue found in the MOL discrete-monitoring code path, and present the new COS-American engine: its numerical design, two genuine numerical instabilities found and fixed during development, and its accuracy against MOL for discretely- and continuously-monitored American barrier options, with and without jumps.

1 The three (now four) engines

<code>mol.py</code>	Finite-difference method of lines: the Heston PDE is discretized along lines of constant variance, the resulting 2nd-order ODE system in the spot variable is solved via a Riccati transform (forward/backward sweep), and the variance lines are coupled by a Gauss–Seidel sweep, all inside an implicit-Euler-then-BDF2 time stepping loop. Handles continuous or discrete monitoring, European or American exercise, and knock-out or (via in-out parity) knock-in payoffs. Price, delta and gamma all come directly out of the ODE system, with no finite differencing. The per-line recursion is Numba-JIT compiled.
<code>cos2d.py</code>	2D Fourier-cosine expansion: variance is discretized on $M=200$ Clenshaw–Curtis quadrature nodes; for each node, a standard discretely-monitored 1D COS recursion (Fang & Oosterlee 2009) runs in the log-price dimension <i>as if</i> variance were frozen at that node for the remaining life of the option, and only the very last step blends the M frozen-variance branches together using the true one-step transition density of variance from today’s v_0 . European, knock-out only, discrete monitoring only.
<code>cos2d.american.py</code>	New. Bermudan-COS extension of the same frozen-variance-branch architecture to American exercise, both discrete- and continuous-barrier monitoring. See Section 4.
<code>mc.py</code>	Monte Carlo simulation of the same Heston(+jump) dynamics (full-truncation Euler scheme for the variance), with a Broadie–Glasserman–Kou continuity correction applied only where the barrier is genuinely continuously monitored. Delta/gamma via common-random-number central bumps. European only.

All four share a common parameter/spec layer (`params.py`) so that the same option and Heston(+jump) specification can be priced by whichever subset of engines supports it.

2 Replication of Chiarella, Kang & Meyer (2012)

Parameters throughout (Table 1 of the paper): $r=0.03$, $q=0.05$, $T=0.5$, $K=100$, $H=130$, $\kappa_v=2.00$, $\theta=0.10$, $\sigma=0.10$, $\rho=-0.50$, $v_0=0.10$, no jumps. Up-and-out call, spot $S_0 \in \{80, 90, 100, 110, 120\}$.

A discrepancy in the paper’s own description, resolved by triangulation

The paper’s Section 4 states discretely-monitored examples use “weekly monitoring frequency”. Taken literally (≈ 26 evenly-spaced dates over $T=0.5y$), *none* of the engines reproduce Tables 4–5 – MOL, COS and MC all independently priced well below the published values, and a Broadie–Glasserman–Kou continuity-correction check confirmed that scale of price is what ~ 26 monitoring dates should actually produce. What *does* reproduce Tables 4–5 almost exactly is monitoring at just the schedule hardcoded in the companion Fortran solvers already in this repository: 2 dates ($T/2, T$) for the European case (`Eur_disc.f90`) and 3 ($T/3, 2T/3, T$) for the American case (`Ame_disc.f90`). We use that schedule below.

Grid: $n_S=150$, $M=150$, 150–300 time steps. **good** match throughout.

MOL: **good** match ($< 0.3\%$ relative throughout), including the free-boundary / early-exercise machinery. **COS (new)**: excellent at-the-money and in-the-money ($< 1.1\%$ from the paper, $S_0 \geq 100$); a real, consistently $\sim$ constant-absolute ($\approx \$0.10$) bias remains for deep out-of-the-money spots, discussed in Section 4.

Table 2: Continuous monitoring, European (no early exercise)

S_0	MOL (this work)	MOL (paper, finest grid)	abs. diff
80	0.9038	0.9046	0.0008
90	1.8747	1.8806	0.0059
100	2.5854	2.5999	0.0145
110	2.4709	2.4858	0.0149
120	1.4740	1.4812	0.0072

Table 3: Continuous monitoring, American (early exercise) – **now with COS**

S_0	COS (py, new)	MOL (this work)	MOL (paper, finest grid)	COS vs. paper
80	1.5095	1.4029	1.4012	+7.73%
90	4.0346	3.9383	3.9364	+2.50%
100	8.3199	8.3051	8.3003	+0.24%
110	14.3002	14.4206	14.4033	−0.72%
120	21.5820	21.8732	21.8219	−1.10%

COS: **good**, max abs. diff 0.007 ($< 0.2\%$) – essentially exact. **MOL / MC:** **fair**, a consistent 2–4% low bias that does not shrink cleanly with grid/path refinement (investigated in Section 3).

MOL: **good** match ($< 0.4\%$ relative), despite using the same discrete-monitoring domain-switch code as Table 4’s European case, which is the puzzle addressed next. **COS (new):** the same pattern as the continuous case above – excellent at $S_0 \geq 100$ ($< 0.8\%$), a real $\sim 8\%$ relative (but modest, $\approx \$0.11$, absolute) gap deep out-of-the-money. The paper itself has no COS column here (it only supports European exercise in the original MATLAB code); this is a new comparison, not a discrepancy against a published number.

3 Investigating the MOL European discrete-monitoring bias

Table 4’s MOL/MC $\sim 2\text{--}4\%$ low bias, and its failure to shrink monotonically under grid refinement ($n_S=75/150/250$ gave 4.02/3.95/4.03 at $S_0=100$), was investigated directly rather than left unexplained.

1. **Ruled out – SOR/Gauss–Seidel under-convergence.** Tightening the sweep tolerance from 10^{-8} to 10^{-10} and the iteration cap from 60 to 150 changed the price by less than 10^{-4} : the per-step nonlinear solve is fully converged; this is not the source.
2. **Ruled out – a monitoring-date/tau convention mismatch.** MOL, COS and MC each convert the shared `OptionSpec.monitor_dates` field via the same $T - \text{monitor_dates}$ expression, but MOL’s own internal time axis runs the *opposite* direction ($\tau=0$ at maturity, $\tau=T$ “now”) from the axis COS/MC use. Testing the corrected input directly reproduced the *identical* prices to four decimal places. This is not the source (though the convention split is real and worth documenting to avoid future confusion).
3. **Leading hypothesis – domain-switch remeshing sensitivity, masked by intrinsic value in the American case.** At each discrete monitoring date, MOL remeshes the price/delta surface from the wide domain $(0, S_{\max})$ onto the narrow domain $(0, H)$ and back

Table 4: Discrete monitoring (2 dates: $T/2, T$), European

S_0	COS (py)	COS (paper)	MOL (py)	MOL (paper)	MC (py)
80	1.0805	1.0809	1.0659	1.0807	1.0557
90	2.4848	2.4871	2.4613	2.5289	2.4401
100	4.0405	4.0454	3.9474	4.1116	3.9728
110	4.9710	4.9779	4.7517	5.0235	4.7723
120	4.8578	4.8646	4.5329	4.8706	4.5563

Table 5: Discrete monitoring (3 dates: $T/3, 2T/3, T$), American – **now with COS**

S_0	COS (py, new)	MOL (this work)	MOL (paper)	COS vs. paper
80	1.5123	1.4120	1.4008	+7.97%
90	4.0439	3.9512	3.9339	+2.80%
100	8.3648	8.3256	8.3010	+0.77%
110	14.4605	14.4826	14.4446	+0.11%
120	22.0256	22.0926	22.0389	−0.06%

via linear interpolation, against a sinh-stretched grid whose exact point placement near $S=H$ shifts non-smoothly as n_S changes. The resulting interpolation error appears comparable in *absolute* size for European and American contracts, but the European price here (≈ 4) has no intrinsic-value floor to absorb it, whereas the American price (≈ 8.3) is dominated by the robust, purely-algebraic exercise region – so the same absolute noise reads as 2–4% for European and $< 0.4\%$ for American. This is consistent with all observations above but was not converted into a code fix in this pass; a natural next step is a higher-order (e.g. cubic) remesh, or local grid refinement pinned exactly at $S=H$ independent of n_S .

4 The new COS-American engine (`cos2d_american.py`)

`cos2d.py`’s discretely-monitored recursion never leaves Fourier space between monitoring dates – the barrier is enforced by restricting an FFT-convolution kernel to the sub-interval (a, h) , a trick that exists specifically to avoid ever evaluating the option value pointwise. American exercise has no such trick available in general: $\max(\text{continuation}, \text{intrinsic})$ is not a linear operation representable as a convolution. Following Fang & Oosterlee (2009), the new module leaves Fourier space at every exercise opportunity – evaluate the continuation value pointwise on a grid, take the max with the intrinsic payoff, and numerically re-expand the result back onto Fourier-cosine coefficients (a forward cosine transform, implemented as an explicit real cosine/sine basis matrix rather than an FFT, for a self-verifiable round trip) before continuing backward. Because American exercise is not naturally discrete the way a monitoring date is, the engine always runs a dense Bermudan grid of `n_exercise` exercise dates and Richardson-extrapolates across increasing `n_exercise` to approximate continuous exercise; “discrete” vs. “continuous” *barrier* monitoring is then just a choice of which of those dates also enforce the knock-out truncation.

Two numerical instabilities found and fixed during development

1. **Chaining the true Heston transition is not a semigroup.** `cos2d.py`'s frozen-variance branches apply the true one-period Heston transition characteristic function once or twice per branch ($mt-1 \leq 2$ in the validated tables above). Chaining that same formula tens of times (needed for a dense Bermudan grid), always re-anchoring to the same starting variance at every fine step, is not a semigroup: $\phi_{\text{Heston}}(u, v, dt)^n \neq \phi_{\text{Heston}}(u, v, n dt)$ (verified numerically – the gap saturates but is non-negligible), and iterating this through the evaluate/reproject round trip tens of times compounded into a $> 15\%$ price bias by 50 steps, growing with the step count. **Fix:** each branch's fine recursion uses a constant-variance Black–Scholes(+jump) characteristic function instead – an exact Lévy semigroup, verified floating-point-exact under chaining, and arguably the more literal reading of “variance frozen for the whole remaining life” anyway.
2. **The final cross-branch blend's quadrature breaks down at fine dt .** `cos2d.py`'s one-shot blend weights branches by the true one-step variance-transition density, evaluated by quadrature over its M -node Clenshaw–Curtis grid – valid because its own $dt=T/mt$ is large. Reusing that blend at this module's tiny fine-step dt was tried first: the transition density's width shrinks below the quadrature grid's local node spacing, its normalization drifts from ≈ 1 to > 2 as dt shrinks, and the resulting price swung non-monotonically by $\sim 10\%$ from changing M alone (75 vs. 300), independent of N or `n_exercise`. **Fix:** since each branch's reconstructed value is already smooth in variance (verified: no node-to-node noise across dense M sweeps), and since variance cannot have moved from v_0 over an infinitesimal step anyway, the final read-out is plain interpolation of the (already exercise/barrier-corrected) per-branch values at the real v_0 – both simpler and the correct $dt \rightarrow 0$ limit. Verified to remove the M -sensitivity entirely (identical to four significant figures across $M=75\text{--}300$).
3. **Continuous monitoring needs N to scale with `n_exercise`.** Discrete barrier schedules truncate the value to zero above h at only 2–3 fixed dates, so the number of times a fresh discontinuity is reprojected through the N -term cosine basis stays small even as `n_exercise` grows – $N=256$ converges cleanly there into the hundreds of exercise dates. Continuous monitoring truncates at *every* exercise node, so that cost scales with `n_exercise` itself: at $N=256$ the $S_0=120$ continuous case converges smoothly only up to `n_exercise` ≈ 320 before visibly diverging (21.83 at 320 $\rightarrow$ 23.18 at 640 $\rightarrow$ 23.92 at 1280, versus a converged value near 21.8). Doubling N to 512 pushed the divergence onset comfortably past the `n_exercise` range needed for good accuracy. This is documented as a usage requirement in the module (not solved with an automatic guard): continuously-monitored runs need a larger N than discretely-monitored ones at the same `n_exercise`.

None of these are subtle – each was caught by a direct numerical experiment (chain the transition analytically and compare; sweep M alone holding everything else fixed; sweep `n_exercise` alone and watch for monotonic convergence) before being accepted as fixed, following the same “trust but verify against a structurally unrelated method” discipline Section 3 above uses for MOL.

5 American exercise with jumps

Merton/Bates log-normal jump parameters $\lambda=0.5$, $\gamma=-0.05$, $\delta=0.1$ (same illustrative values as the package's own quick-start example), otherwise the Table 3/5 parameters above. `cos2d_american.py`

inherits `cos2d.py`'s jump characteristic-function convention (the log-jump-size mean is shifted *inside* the CF itself, no separate drift compensator), which is *not* interchangeable with `charfunc.py`'s compensated convention that `mol.py` uses – see `cos2d.py`'s docstring for why (a calibration-locked choice, not something to "fix"). So the comparison below validates that the American-exercise/barrier machinery behaves correctly with jumps active – sensible, stable numbers, same accuracy pattern as the no-jump case – not exact cross-engine agreement, which the convention difference alone would preclude regardless of any other error.

Table 6: American barrier with jumps ($\lambda=0.5, \gamma=-0.05, \delta=0.1$)

S_0	COS+jump (py)	MOL+jump (py)	diff
<i>Discrete monitoring (3 dates: $T/3, 2T/3, T$)</i>			
80	1.4686	1.3840	+6.11%
100	8.1009	8.1049	-0.05%
120	21.5928	21.7190	-0.58%
<i>Continuous monitoring</i>			
80	1.4637	1.3780	+6.22%
100	8.0336	8.0841	-0.62%
120	21.2296	21.5864	-1.65%

The pattern is the same one seen throughout Section 4: excellent agreement at- and in-the-money, a larger (but stable, non-diverging) gap deep out-of-the-money, on top of the expected small residual from the jump-CF convention difference. Nothing here suggests the jump term interacts badly with the exercise/barrier machinery – the deep-OTM gap size and direction are consistent with the no-jump case, not amplified by adding jumps.

6 Summary

The COS method, previously restricted to European discretely-monitored barrier options in this codebase, now also prices American exercise (discrete or continuous barrier, with or without jumps) via a Bermudan-COS recursion, at a level of accuracy (excellent ATM/ITM, a modest deep-OTM bias) and speed (still faster than MOL, though not by the $\sim 100\times$ margin of the pure-European case, since American exercise requires leaving Fourier space at every one of many exercise dates rather than a handful of monitoring dates) that makes it a genuine second, structurally-independent method for American barrier options under Heston(+jumps) – alongside, not a replacement for, `mol.py`, which remains the more accurate choice deep out-of-the-money until that residual bias is root-caused.